PIKVA · VOLUME 1

Backend Technical Manual

Every file, import, function, dependency and workflow of the Pikva FastAPI backend, explained from first principles for someone new to software.

Codebase: **KIDSPICKUP/BACKEND** (Python 3.14 · FastAPI · PostgreSQL 16 · Redis 7)

State described: commit 7d18594, 16 October 2026

Companion: Volume 2, Frontend Technical Manual (KIDSPICKUP/FRONTEND)

Source of truth for requirements: PIKVA_SRS.md (sections are cited as "SRS 9.2")

Contents

Part A · Foundations

1. How to read this manual
2. What Pikva does
3. Core ideas in plain English
4. The technology stack
5. Folder map and architecture

Part B · File by file

6. How to read Python in this codebase
7. app/main.py: building the application
8. app/core/config.py: settings
9. app/core/constants.py: tuning numbers
10. app/core/security.py: passwords, tokens and codes
11. app/core/dependencies.py: who is calling?
12. app/core/rate_limit.py: counting attempts in Redis
13. app/core/logging_config.py
14. app/db: database and Redis plumbing
15. app/models: the database tables
16. app/schemas: what goes in and out
17. app/routes: the HTTP endpoints
18. app/services: the business rules
19. Alembic: migrations
20. The test suite

Part C · End-to-end workflows

21. A school signs up
22. Signing in and using the API
23. An admin adds a parent (or gate staff)
24. Adding a pupil
25. A parent adds a pickup person; the school approves
26. A parent gets today's pickup code
27. At the gate: verify, then release or refuse
28. After the pickup: history, alerts and audit
29. Password reset, password change and deactivation
30. Security measures at a glance

Part D · Owning the code

31. Running the backend locally
32. Running the tests
33. Recipe: adding a feature safely
34. Deploying
35. Troubleshooting

36. Glossary

Part A · Foundations

What Pikva is, the ideas you need, and the tools it is built with

1. How to read this manual

This manual is written so that you can **own** the backend: understand why each piece exists, change it safely, and explain it to someone else. It assumes no prior experience. Every technical word is explained the first time it appears, and there is a glossary at the end.

It is organised in four parts:

- **Part A, Foundations:** what the product does, the basic ideas behind web backends, the technology stack and how the folders fit together.
- **Part B, File by file:** every Python file in `BACKEND/app`, with its imports, classes and functions explained in order.
- **Part C, End-to-end workflows:** real journeys (a school signs up, a parent gets a pickup code, the gate releases a child) traced through every layer, from HTTP request to database row and back.
- **Part D, Owning the code:** how to run it, test it, change it, deploy it and troubleshoot it, plus the glossary.

Coloured boxes appear throughout:

> **IN PLAIN ENGLISH**
> A plain-language restatement of a technical idea.

> **WHY IT IS BUILT THIS WAY**
> The reasoning behind a design decision, so you can judge whether to keep it.

> **WATCH OUT**
> A trap to avoid when you change this part of the code.

> **TRY IT YOURSELF**
> A small exercise you can run on your own machine to see the idea working.

2. What Pikva does

Pikva is a **secure school pickup verification platform**. The problem it solves (SRS 2.1): at the end of the school day, staff at the school gate must hand each child only to an adult the parent has approved. Paper lists and phone calls are slow and easy to fool. Pikva replaces them with a verifiable, recorded process.

The three kinds of user (roles)

Role	Value stored in the database	What they do
School administrator	`school_admin`	Registers the school, creates parent and gate staff accounts, manages pupils, approves pickup people, reviews security alerts and the audit log.
Parent	`parent`	Sees their own pupils, adds the people allowed to collect them ("pickup people"), and requests a one-time daily pickup code for one of those people.
Gate staff	`gate_staff`	At the gate, types or scans the code the adult presents, checks the photo and name the system shows, then releases the pupil or refuses.

The pickup story in one paragraph

A parent adds "Uncle Obi" as a pickup person for their child. The school administrator approves Uncle Obi. On the day, the parent asks Pikva for **today's pickup code** for Uncle Obi; Pikva generates a random 8-character code (and a QR code), shows it to the parent once, and stores only a scrambled fingerprint (a *hash*) of it. The parent sends the code to Uncle Obi. At the gate, staff type or scan the code; Pikva checks it and shows the pupil's and Uncle Obi's details. If the person matches, staff press **Release**: the code is marked used and a permanent release record is written. Any failed check raises a **security alert** and every important action is written to an **audit log**.

Multi-school ("multi-tenant") design

One Pikva installation serves many schools. Every user, pupil, alert and audit record belongs to exactly one school (`school_id`), and every query the backend runs is filtered by the signed-in user's school. A user can never see another school's data. This is called **tenant isolation** (SRS 21.2) and you will see it in nearly every service function.

3. Core ideas in plain English

Before reading the code, these ideas need to be clear. Skim the ones you already know.

Client, server and backend

A **server** is a program that waits for requests and answers them. A **client** is a program that sends requests (a web browser, a mobile app, or another server). The Pikva **backend** is the server program in `BACKEND/`. It holds all the rules and all the data. The **frontend** (Volume 2) is the website people see; it never decides anything important on its own, it always asks the backend.

> **IN PLAIN ENGLISH**
>
> The backend is the school office: it keeps the records and makes decisions. The frontend is the reception desk window: it shows information and passes requests through, but cannot approve anything itself.

HTTP: requests and responses

Clients and servers talk using **HTTP**. A request has:

- a **method** (the kind of action): `GET` read, `POST` create or perform an action, `PATCH` change part of something, `DELETE` remove;
- a **path**, such as `/api/v1/pupils/7`;
- **headers**, small labelled values such as `Authorization: Bearer <token>`;
- an optional **body**, usually JSON.

The response has a **status code** and usually a JSON body. The status codes Pikva uses:

Code	Meaning	Example in Pikva
200 OK	Worked; here is the answer	List of pupils
201 Created	A new record was made	New pupil, new pickup code, pupil released
202 Accepted	Request taken; work continues later	Password reset email requested
204 No Content	Worked; nothing to return	Password reset confirmed
400 Bad Request	The request makes no sense now	Reset link invalid, wrong current password
401 Unauthorized	You are not signed in (or your token is bad)	Expired token
403 Forbidden	Signed in, but your role may not do this	A parent calling an admin endpoint
404 Not Found	No such record (or not yours to see)	Another school's pupil
409 Conflict	Clashes with current state	Email already used; alert already resolved
422 Unprocessable	The data is invalid	Missing name, end time before start time
429 Too Many Requests	Rate limit hit; wait	Too many login attempts
503 Service Unavailable	A dependency is down	`/health/redis` when Redis is off

JSON

JSON (JavaScript Object Notation) is the text format used for request and response bodies: `{"first_name": "Chidi", "date_of_birth": "2016-05-01"}`. Curly braces hold named values; square brackets hold lists.

API and endpoint

An **API** (Application Programming Interface) is the set of requests a server accepts. Each combination of method and path is an **endpoint**, for example `POST /api/v1/pickup/verify`. Pikva's API is versioned: everything lives under `/api/v1` so a future `/api/v2` can change shapes without breaking old clients (SRS 17.1).

Database, tables, rows, keys

A **relational database** (Pikva uses PostgreSQL) stores data in **tables**, like spreadsheets. Each **row** is one record (one pupil); each **column** is one property (first name). Every row has a unique **primary key** (`id`). A **foreign key** is a column that points at another table's row, for example `pupils.parent_id` points at `users.id`. The database refuses a foreign key that points at nothing, which keeps data consistent. An **index** is a lookup structure that makes searching a column fast, like a book's index.

A **transaction** groups several changes so that either all happen or none do. "Commit" makes them permanent; "rollback" undoes them.

SQL and the ORM

SQL is the language databases understand (`SELECT * FROM pupils WHERE school_id = 3`). Writing SQL strings by hand is error-prone, so Pikva uses an **ORM** (Object-Relational Mapper), **SQLAlchemy**. Each table is described as a Python class (a **model**), each row becomes a Python object, and queries are built with Python code that SQLAlchemy turns into safe SQL.

Migrations

When the shape of a table changes (a new column), the live database must be changed too, without losing data. A **migration** is a small script that applies one such change (`upgrade`) and can undo it (`downgrade`). **Alembic** runs them in order and records which ones have been applied.

Synchronous vs asynchronous (async)

A server spends most of its time *waiting*: for the database, for Redis, for an email service. In **async** code (`async def` and `await`), while one request waits, the server works on another. Pikva is fully async: FastAPI, the database driver (asyncpg), SQLAlchemy's async session and the Redis client all support it. When you see `await`, read it as "pause here until this finishes, and let other work run meanwhile".

Hashing, HMAC and encryption

A **hash** turns text into a fixed-length fingerprint that cannot be turned back. Pikva never stores passwords, pickup codes or reset tokens themselves, only hashes. To check one, it hashes what was typed and compares fingerprints.

- **Argon2** (passwords) is deliberately slow and memory-hungry, so an attacker who steals the database cannot try billions of guesses.
- **HMAC-SHA256** (pickup codes) is a hash mixed with a secret key. Codes are only 8 characters, so a plain hash could be reversed by trying every code; the secret key prevents that.
- **SHA-256** (reset tokens) is a fast plain hash, safe here because reset tokens are 32 random bytes and impossible to guess.

Encryption is different: it can be reversed with a key. Pikva does not need encryption for these values because it never needs the originals back.

Authentication, authorisation and JWT

Authentication answers "who are you?" (signing in). **Authorisation** answers "are you allowed to do this?" (roles and school boundaries). After a successful login Pikva gives the client a **JWT** (JSON Web Token): a small signed text such as `eyJhbGc...` that says "this is user 42, issued at 10:00, expires at 10:30". The signature uses the server's `SECRET_KEY`, so nobody can forge or edit a token. The client sends it on every request in the header `Authorization: Bearer <token>`.

Redis and rate limiting

Redis is a very fast in-memory key-value store. Pikva uses it for counters that expire on their own: "this IP has tried to log in 7 times in the last 15 minutes". **Rate limiting** means refusing requests (HTTP 429) once a counter passes a limit, which stops password guessing and code guessing.

Environment variables and configuration

Settings that differ between your laptop and the real server (database password, secret key) are not written in the code. They come from **environment variables**, usually loaded from a file named `.env` that is never committed to git.

Dependency injection

FastAPI lets an endpoint declare what it needs (`session: AsyncSession = Depends(get_session)`) and FastAPI provides it. This is called **dependency injection**. Pikva uses it for the database session, the Redis client, the signed-in user, role checks and rate limits. It keeps endpoint code short and makes testing easy.

Background tasks

Some work, like sending an email, should not delay the answer. FastAPI's `BackgroundTasks` runs a function *after* the response has been sent.

4. The technology stack

These are declared in `BACKEND/pyproject.toml`; exact versions are pinned in `BACKEND/uv.lock`.

Tool / library	What it is	How Pikva uses it
Python 3.14	The programming language.	All backend code. `requires-python = ">=3.14"`.
uv	A fast Python package and project manager.	Installs dependencies (`uv sync`) and runs commands in the project's environment (`uv run pytest`).
FastAPI (`fastapi[standard]`)	A web framework for building APIs with type hints.	Routing, request validation, dependency injection, background tasks, automatic Swagger docs at `/docs`.

Uvicorn (via fastapi[standard])	An ASGI server: the program that listens on a network port and hands requests to FastAPI.	Started by `fastapi dev` / `fastapi run`.
Pydantic (via FastAPI)	Data validation using Python type hints.	Every request and response shape in `app/schemas`. Bad input becomes a 422 automatically.
pydantic-settings (via fastapi[standard])	Reads settings from environment variables / `.env`.	`app/core/config.py`.
email-validator (via fastapi[standard])	Checks email address syntax.	Powers Pydantic's `EmailStr` type.
httpx (via fastapi[standard])	An async HTTP client.	Sends emails to the Resend API; the test suite uses it to call the app in-process.
SQLAlchemy 2.x (`[asyncio]`)	The ORM and SQL toolkit.	Models in `app/models`; all queries in `app/services`.
asyncpg	A fast async PostgreSQL driver.	The connection layer under SQLAlchemy (`postgresql+asyncpg://` URL).
Alembic	Database migration tool for SQLAlchemy.	`BACKEND/alembic/versions/*.py`; `uv run alembic upgrade head`.
pwdlib[argon2]	Password hashing library (argon2-cffi underneath).	`hash_password` / `verify_password` in `security.py`.
PyJWT	Creates and verifies JSON Web Tokens.	`create_access_token` / `decode_access_token`, algorithm HS256.
redis (redis-py, async)	Python client for Redis.	Rate limits and email cooldowns.
PostgreSQL 16	The relational database server.	All permanent data. Runs in Docker locally.
Redis 7	In-memory key-value server.	Short-lived counters only; persistence is switched off.
Docker Compose	Runs services in containers from one file.	`docker-compose.yml` starts PostgreSQL and Redis locally.
Resend	An email-sending web service.	Security alerts, approval requests, account setup and password emails. Optional in development (emails are logged instead).
pytest + **pytest-asyncio** (dev)	Test runner and its async plugin.	`BACKEND/tests`; `uv run pytest`.

Python's standard library is also used heavily: `asyncio` (async runtime and worker threads), `datetime` (timestamps, always UTC), `hashlib` / `hmac` / `secrets` (hashing and secure randomness), `logging`, `json`, `dataclasses`, `enum` and `typing`.

pyproject.toml, line by line

```
[project]
name = "backend"
version = "0.1.0"
requires-python = ">=3.14"          # refuse older Pythons
dependencies = [                     # installed everywhere, including production
    "alembic>=1.20.0", "asyncpg>=0.31.0", "fastapi[standard]>=0.141.1",
    "pwdlib[argon2]>=0.3.1", "pyjwt>=2.15.1", "redis>=5",
    "sqlalchemy[asyncio]>=2.1.3",
]
[dependency-groups]
dev = ["pytest>=9.1.1", "pytest-asyncio>=1.4.0"]   # only for development and tests

[tool.pytest.ini_options]
testpaths = ["tests"]                # where tests live
pythonpath = ["."]                   # so "import app..." works in tests
asyncio_mode = "auto"                # every async test runs without a decorator
asyncio_default_fixture_loop_scope = "session"   # one event loop shared by all tests,
asyncio_default_test_loop_scope = "session"      # matching the app's shared DB engine
```

The text in square brackets, such as `[asyncio]`, selects an **extra**: optional parts of a package. `>=` means "this version or newer"; the lock file pins the exact version actually installed.

docker-compose.yml

Starts two containers for local development:

- **postgres** (`postgres:16`): database `pikva`, user/password `postgres`/`postgres` (development only), port 5432, data kept in the named volume `pikva_postgres_data` so it survives restarts.
- **redis** (`redis:7-alpine`): started with `--save "" --appendonly no` so nothing is written to disk (counters expire anyway), and bound to `127.0.0.1:6379` so only your own machine can reach it.

.env.example: every setting

Variable	Purpose
`ENVIRONMENT`	`development` or `production`. Production hides `/docs` and refuses a weak `SECRET_KEY`.
`DATABASE_URL`	Where PostgreSQL is, e.g. `postgresql+asyncpg://pikva:pw@127.0.0.1:5432/pikva`.
`SECRET_KEY`	Signs JWTs and derives the pickup-code key. At least 32 characters. Changing it signs everyone out and invalidates today's codes.
`ACCESS_TOKEN_EXPIRE_MINUTES`	How long a login lasts (default 30).
`DATABASE_ECHO`	`true` prints every SQL statement (debugging only).
`REDIS_URL`	Redis address. Database number 15 is reserved for tests.
`LOG_LEVEL`	`DEBUG`, `INFO`, `WARNING` or `ERROR`.
`RESEND_API_KEY`, `EMAIL_FROM`	Email sending. If either is empty, emails are only logged.

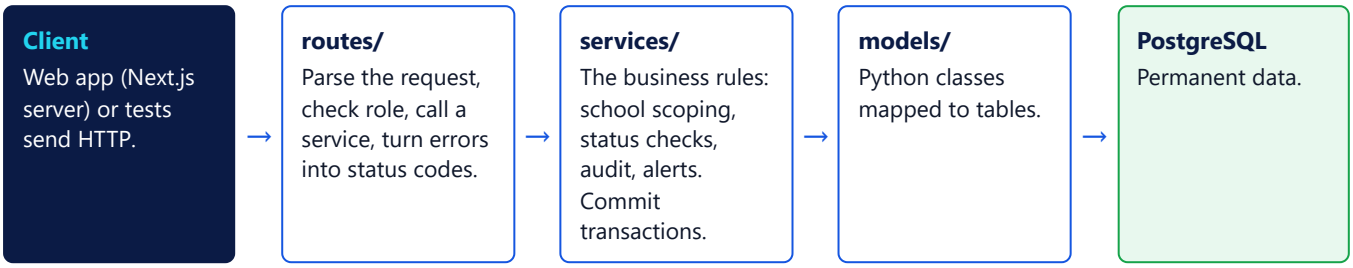
`WEB_APP_URL`	Public address of the web app, used in links inside emails.

5. Folder map and architecture

```
BACKEND/
├── pyproject.toml, uv.lock      # dependencies
├── docker-compose.yml           # local PostgreSQL + Redis
├── .env.example                 # settings template (copy to .env)
├── alembic.ini, alembic/        # database migrations
│   ├── env.py                   # how Alembic connects
│   └── versions/*.py            # one file per schema change (10 files)
├── app/
│   ├── main.py                  # creates the FastAPI app
│   ├── core/                    # cross-cutting: config, security, auth, rate limits, logging
│   ├── db/                      # database engine/session, Redis client, model base class
│   ├── models/                  # one class per database table
│   ├── schemas/                 # request/response shapes (Pydantic)
│   ├── routes/                  # HTTP endpoints, grouped by feature
│   ├── services/                # business rules and all database work
│   └── repositories/            # empty placeholders (see below)
├── tests/                       # 40+ pytest files
├── scripts/e2e_school.py        # throwaway school for browser tests
├── DEPLOYMENT.md                # production checklist
└── MANUAL_TESTING.md            # step-by-step manual checks
```

The layers and the rule between them

The code is arranged in layers. Each layer only calls the layer below it.



schemas/ sit beside the routes (they define what goes in and out), and **core/** and **db/** are shared tools every layer may use.

> **WHY IT IS BUILT THIS WAY**
>
> Routes stay thin (HTTP concerns only) and services hold the rules. That means the same rule (for example "a parent may only see their own pupils") is written once in a service and reused by every endpoint, and services can be tested without HTTP. The SRS (section 16) asks for this separation.

IN PLAIN ENGLISH

The `repositories/` files and `services/picker_service.py` and `schemas/picker.py` exist but are empty. They were planned placeholders; the work they would hold lives in the services instead. You may delete them or leave them; nothing imports them.

The life of one request

Here is what happens, in order, when the web app asks for a pupil list with `GET /api/v1/pupils` and a Bearer token:

1. **Uvicorn** receives the bytes on port 8000 and hands an ASGI request to the FastAPI app made in `main.py`.
2. **FastAPI** matches the path to `list_pupils_endpoint` in `routes/pupils.py` (registered under the `/api/v1` prefix by `routes/__init__.py`).
3. **Dependencies** run first: `get_session` opens a database session; `require_role(PARENT, SCHOOL_ADMIN)` calls `get_current_user`, which reads the Bearer token, verifies its signature and expiry, loads the user row, and checks the account is active and the token is newer than the last password change. Any failure stops here with 401 or 403.
4. **Route** calls `list_pupils(session, school_id, parent_id)`, where `parent_id` is the user's own id for parents and `None` for admins.
5. **Service** builds a query limited to the school (and parent), with the parent's details loaded, ordered by name, and awaits the result from PostgreSQL.
6. **FastAPI** converts the model objects to JSON using `response_model=list[PupilResponse]`, which also guarantees no extra fields (like password hashes) can leak.
7. **Session** closes when the request ends (the `async with` in `get_session`).

Part B · File by file

Every module in BACKEND/app: imports, classes and functions, in the order you meet them

6. How to read Python in this codebase

A few language features appear everywhere. Learn these once:

You see	It means
`from x import y`	Bring the name `y` from module `x` into this file. `app.core.config` is the file `app/core/config.py`.
`def f(a: int) -> str:`	A function. `a: int` is a **type hint** (a is expected to be an integer); `-> str` says it returns text. Hints are not enforced by Python itself, but FastAPI and Pydantic read them to validate data.
`async def` / `await`	An asynchronous function, and "wait for this without blocking the server".
`int | None`	"An integer, or nothing". `None` is Python's "no value".
`"""text"""` under a def	A **docstring**: documentation for the function. Pikva has one on nearly every function.
`@something` above a def	A **decorator**: wraps or registers the function. `@router.get("/x")` registers an endpoint.
`class X(Base):`	A class `X` that inherits behaviour from `Base`.
`@dataclass`	A simple class that just holds named values.
`class R(str, Enum)`	An **enumeration**: a fixed set of allowed values that also behave as text.
`raise ValueError("...")`	Stop and report an error. Services raise `ValueError`; routes catch it and choose an HTTP status.
`try: ... except E:`	Run code; if error type `E` happens, handle it instead of crashing.
`async with x as y:`	Open something (a session, a pipeline) and close it automatically at the end of the block.
`f"Hello {name}"`	An f-string: text with values inserted.
`_name`	A leading underscore means "private to this file; do not import it elsewhere".

7. app/main.py: building the application

`app/main.py`

This is the entry point. When you run `uv run fastapi dev app/main.py`, the server imports this file and looks for the variable `app`.

Imports

Import	Why
`logging`	To log a warning about a weak key in development.
`asynccontextmanager` (contextlib)	Turns `lifespan` into a start-up/shut-down hook.
`FastAPI`	The application class.
`secret_key_weakness, settings`	The loaded configuration and the key checker (`core/config.py`).
`configure_logging`	Sets up log output (`core/logging_config.py`).
`RateLimitExceeded, rate_limit_exceeded_handler`	The rate-limit error and the function that turns it into a 429 response.
`close_redis`	Closes Redis connections on shutdown.
`router` (app.routes)	All endpoints, already grouped under `/api/v1`.

`check_secret_key()`

Asks `secret_key_weakness` whether `SECRET_KEY` is weak. If it is fine, returns. If weak and the environment is production, raises `RuntimeError`, so the server **refuses to start** with a message telling you how to generate a good key. In development it only logs a warning.

`lifespan(app)`

Code before `yield` runs at start-up (nothing here); code after runs at shutdown: `await close_redis()` closes the shared Redis connections cleanly.

`create_app()`

1. `configure_logging(settings.log_level)`: sets up logging first so every later message is formatted.
2. `check_secret_key()`.
3. Creates `FastAPI(...)` with a title, description and version for the docs, the lifespan hook, and `docs_url` / `redoc_url` / `openapi_url` set to `None` in production, which hides the interactive API documentation from the public (SRS 24).
4. `app.add_exception_handler(RateLimitExceeded, ...)`: whenever any code raises `RateLimitExceeded`, answer 429 with a `Retry-After` header.
5. `app.include_router(router)`: attaches every endpoint.

The last line, `app = create_app()`, runs all of this when the module is imported.

> **WHY IT IS BUILT THIS WAY**
>
> Wrapping set-up in a function (an "app factory") lets tests and production build the app the same way, and keeps the production safety checks impossible to skip.

8. app/core/config.py: settings

`app/core/config.py`

Imports

`Literal` (typing) restricts a value to listed options. `SecretStr` (pydantic) hides a value when printed (shows `**********`), used for the Resend API key. `BaseSettings` and `SettingsConfigDict` (pydantic_settings) read environment variables.

`class Settings(BaseSettings)`

Each attribute is a setting. Pydantic-settings fills it from the environment variable with the same name in capitals (`database_url` ← `DATABASE_URL`), reading `.env` as well (`model_config = SettingsConfigDict(env_file=".env")`). Attributes without a default (`database_url`, `secret_key`) are required: the app fails at start-up if they are missing.

Attribute	Type / default	Note
`environment`	`Literal["development","production"]` = development	A typo such as "prod" fails start-up rather than silently running as development.
`log_level`	Literal of four levels = INFO	
`database_url`, `secret_key`	`str`, required	
`access_token_expire_minutes`	`int` = 30	JWT lifetime.
`database_echo`	`bool` = False	Print SQL.
`redis_url`	`str` = `redis://127.0.0.1:6379/0`	Tests switch to database 15.
`resend_api_key`	`SecretStr | None`	Email; None means "log emails instead".
`email_from`	`str | None`	Sender address on a Resend-verified domain.
`web_app_url`	`str` = `http://localhost:3000`	Used to build links in emails.

`settings = Settings()` creates the single shared settings object that every other module imports.

`secret_key_weakness(secret_key) -> str | None`

Returns a reason the key is weak, or `None`. Checks in order: looks like a placeholder (exactly "changeme", "secret", "password", or contains "change-me", "your-secret", "replace-me"...); shorter than 32 characters; fewer than 10 different characters. The reason never contains the key itself, so it is safe to log.

9. app/core/constants.py: tuning numbers

`app/core/constants.py`

All limits in one place, so they are easy to find and change.

Constant	Value	Meaning
`PICKUP_CODE_ALPHABET`	`23456789ABCDEFGHJKLMNPQRSTUVWXYZ`	32 characters; 0, O, 1 and I are left out because they are easy to confuse when read aloud.
`PICKUP_CODE_LENGTH`	8	32^8 ≈ 1.1 trillion possible codes.
`LOGIN_WINDOW_SECONDS` / `LOGIN_FAILURE_LIMIT` / `LOGIN_IP_LIMIT`	900 / 5 / 30	In 15 minutes: an email is locked after 5 failed logins; an IP may try 30 times.
`GATE_WINDOW_SECONDS` / `GATE_STAFF_LIMIT`	60 / 30	Each gate staff member: 30 verify/refuse/release calls per minute in total.
`CODE_REQUEST_WINDOW_SECONDS` / `CODE_REQUEST_LIMIT`	3600 / 10	Each parent: 10 pickup code requests per hour.
`REGISTER_WINDOW_SECONDS` / `REGISTER_IP_LIMIT`	3600 / 5	5 school registrations per IP per hour.
`PASSWORD_RESET_TOKEN_MINUTES`	30	Reset and set-up links expire after 30 minutes.
`PASSWORD_RESET_*` limits	10/IP, 3/email per hour; confirm 10/IP per 15 min	Stops inbox flooding and token guessing.

10. app/core/security.py: passwords, tokens and codes

`app/core/security.py`

Imports

`asyncio` (to run slow hashing in a thread), `PasswordHash` (pwdlib), `datetime/timedelta/timezone` (token times), `hashlib` and `hmac` (SHA-256 and keyed hashing), `secrets` (cryptographically secure randomness; never use `random` for security), `jwt` (PyJWT), `settings` and the code alphabet/length constants.

`password_hash = PasswordHash.recommended()` creates a hasher with pwdlib's recommended Argon2 settings. Tests replace it with a cheaper one for speed.

Function	What it does and why
`async hash_password(password)`	Returns an Argon2 hash. Argon2 takes about 80 ms of CPU and 64 MiB of memory by design, so it runs in a worker thread (`asyncio.to_thread`) to keep the server free for other requests, such as the gate.

`async unusable_password_hash()`	Hashes a random secret nobody is told. Used for accounts an admin creates: the person cannot sign in until they set their own password through the emailed link.
`async verify_password(password, hashed)`	Checks a typed password against a stored hash, also in a thread. Returns True/False.
`async verify_dummy_password(password)`	Does a full-cost check against a throwaway hash when someone logs in with an email that does not exist. Otherwise unknown emails would fail faster than wrong passwords, and an attacker could time responses to discover which emails have accounts. The dummy hash is made once and reused (`global _dummy_hash`).
`create_access_token(subject)`	Builds a JWT with `sub` (the user id as text), `iat` (issued at) and `exp` (expiry = now + 30 minutes), signed with `SECRET_KEY` using HS256 (HMAC-SHA256).
`decode_access_token(token)`	Verifies the signature and the expiry and returns the payload dictionary. Raises a `jwt.PyJWTError` if either is wrong. Only `HS256` is accepted, which blocks "algorithm confusion" attacks.
`_pickup_code_key`	A second key derived from `SECRET_KEY` with HMAC and the label `pikva:pickup-code` . Keeping separate keys for separate jobs is a standard practice: the JWT signer and the code hasher never share a key.
`generate_pickup_code()`	Picks 8 characters at random from the alphabet with `secrets.choice` .
`hash_pickup_code(code)`	Removes spaces, upper-cases (so "7f3b 9k2d" equals "7F3B9K2D"), and returns the HMAC-SHA256 hex digest with the pickup-code key. This digest is what is stored and looked up.
`generate_password_reset_token()`	32 random bytes as URL-safe text (43 characters).
`hash_password_reset_token(token)`	Plain SHA-256 hex. Safe without a key because the token is unguessable.

WATCH OUT

Changing `SECRET_KEY` changes both the JWT key and the pickup-code key: every user is signed out and every code issued today stops matching. Only rotate it deliberately.

11. app/core/dependencies.py: who is calling?

`app/core/dependencies.py`

Imports `jwt` (for its error type), FastAPI's `Depends` , `HTTPException` , `status` and `HTTPBearer` , SQLAlchemy's `select` and `AsyncSession` , `decode_access_token` , `get_session` , and the `User` model and `UserRole` .

`bearer_scheme = HTTPBearer()` is a FastAPI helper that reads the `Authorization: Bearer ...` header (answering 401/403 automatically if it is missing) and also makes the "Authorize" button appear in Swagger.

`async get_current_user(credentials, session)`

The gatekeeper used by every protected endpoint. Step by step:

1. Takes the token text from the header.
2. Decodes it; bad signature or expired → **401 "Invalid or expired token"**.

3. Reads `sub` and converts it to an integer user id; anything odd → 401.
4. Loads that user from the database; missing → 401 "User not found".
5. If the token was issued before the user's last password change → 401. This is how "sign out everywhere" works without storing sessions.
6. If the account is deactivated → 401. A deactivated user's open sessions stop at once.
7. Returns the `User` object; the endpoint receives it as `current_user`.

`issued_before_password_change(payload, user)`

Compares the token's `iat` (whole seconds) with `password_changed_at` rounded down to whole seconds. A token with no `iat` counts as older. Rounding avoids rejecting a login made in the same second as a reset.

`require_role(*allowed_roles)`

A **dependency factory**: it returns a new dependency function, `role_checker`, which gets the current user and raises **403** unless the user's role is in the allowed list. Used like `dependencies=[Depends(require_role(UserRole.SCHOOL_ADMIN))]`.

> **IN PLAIN ENGLISH**
>
> Every protected endpoint first asks: "Show me your badge" (401 if none or fake), then "Is your badge the right colour for this door?" (403 if not).

12. app/core/rate_limit.py: counting attempts in Redis

`app/core/rate_limit.py`

Implements a **fixed-window** counter: the first hit on a key starts a timer (the window); every hit adds one; when the key expires the count starts again. If Redis is unreachable the limiter **fails open**: it allows the request and logs a warning, so a Redis outage never locks gate staff out.

Imports: `hashlib` (hash emails used in keys), `logging`, FastAPI's `Request` and `JSONResponse`, the async `Redis` client and `RedisError`.

Name	Role
`RATE_LIMITED_RESPONSE`	A description of the 429 response used in route decorators, so Swagger documents it.
`class RateLimitExceeded(Exception)`	The error raised when over the limit; carries `retry_after` seconds.
`hashed(value)`	SHA-256 of a trimmed, lower-cased email, so Redis never holds personal data in plain text.
`_scope(key)`	Extracts the limit name (e.g. `login:email`) from a key like `pikva:rl:login:email:<hash>`, for logging without the subject.
`_blocked(key, retry_after)`	Logs which limit blocked and builds the exception.
`_log_outage(exc)`	Logs that Redis was unavailable, by error type only.

`async _count(redis, key, window)`	In one Redis **transaction pipeline**: `INCR key`, `EXPIRE key window NX` (set the timer only on the first hit), `TTL key`. Returns (count, seconds left).
`async hit(redis, key, limit, window)`	Count this attempt; raise if the count is now above the limit. Used where *every* attempt counts (IP limits, gate, code requests).
`async record(redis, key, window)`	Count without blocking. Used after a *failed* login.
`async check(redis, key, limit)`	Read the count without adding; raise if it has reached the limit. Used before a login to block a locked email.
`async rate_limit_exceeded_handler`	Registered in `main.py`: returns 429 `{"detail": "Too many requests. Try again later."}` with `Retry-After`.

WHY IT IS BUILT THIS WAY

Login uses `check` before and `record` after a failure, so **only failed** logins lock an email. A legitimate user who signs in correctly is never counted against the per-email lock.

13. app/core/logging_config.py

`app/core/logging_config.py`

`configure_logging(level, stream=None)` attaches one handler named `pikva` to the `app` logger (every module's `logging.getLogger(__name__)` starts with `app.`, so they all inherit it). The format is `time LEVEL logger.name: message`. Calling it twice replaces the old handler, so lines are never duplicated. Uvicorn's own loggers are left alone.

WATCH OUT

The code is careful never to log secrets: no passwords, codes, tokens, emails or message bodies; only ids, result names and error type names (SRS 28). Keep that rule when you add log lines.

14. app/db: database and Redis plumbing

`app/db/base.py`

`class Base(DeclarativeBase)` is the parent class of every model. SQLAlchemy uses it to collect all tables into `Base.metadata`, which Alembic and the tests read.

`app/db/session.py`

- `engine = create_async_engine(settings.database_url, echo=settings.database_echo)`: the shared **connection pool** to PostgreSQL. Creating connections is slow, so the pool keeps some open and reuses them.

- `async_session = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)`: a factory that makes sessions. `expire_on_commit=False` keeps loaded objects usable after a commit (otherwise reading an attribute after commit would trigger a new query, which async code cannot do implicitly).
- `async get_session()`: a FastAPI dependency that opens a session for one request and closes it afterwards (`yield` inside `async with`).

> **IN PLAIN ENGLISH**
>
> A **session** is a short conversation with the database for one request. You stage changes (`session.add`), send them without finalising (`flush`, which also gives new rows their ids), and finalise (`commit`) or abandon (`rollback`) them.

app/db/redis.py

`redis_client = Redis.from_url(settings.redis_url, decode_responses=True, socket_connect_timeout=0.5, socket_timeout=0.5)`: one shared client that connects lazily (importing the app never needs Redis running) and gives up after half a second, so a dead Redis fails fast instead of freezing the gate. `get_redis()` provides it to endpoints; `close_redis()` closes it on shutdown.

15. app/models: the database tables

Each file defines one table as a Python class. They all share the same building blocks, imported from SQLAlchemy:

Building block	Meaning
`__tablename__ = "pupils"`	The real table name in PostgreSQL.
`id: Mapped[int] = mapped_column(primary_key=True)`	A column. `Mapped[int]` is the Python type; `mapped_column(...)` holds database options.
`Mapped[str | None]` + `nullable=True`	The column may be empty (NULL).
`String(150)`	Text up to 150 characters (the database rejects longer).
`DateTime(timezone=True)`	PostgreSQL `timestamptz`: a moment in time, always stored in UTC. Every timestamp in Pikva uses this.
`Date`	A calendar date without time (date of birth, code date).
`ForeignKey("schools.id")`	Must point at an existing row of another table.
`default=lambda: datetime.now(timezone.utc)`	Filled in automatically when a row is created. `onupdate=` also refreshes it on every change.
`__table_args__ = (Index(...),)`	Extra indexes for fast lookups; `unique=True` also forbids duplicates.
`JSON`	A column holding a JSON document (audit metadata).

How the tables relate

```
schools ─┬─< users                (school_id)        a user belongs to one school
         ├─< pupils               (school_id)
         ├─< security_alerts      (school_id)
         └─< audit_logs           (school_id)
users   ─┬─< pupils                (parent_id)        a parent has many pupils
         ├─< release_records      (verified_by)      the gate staff member who released
         ├─< password_reset_tokens(user_id)
         ├─< security_alerts      (attempted_by)
         └─< audit_logs           (user_id)          who did it
pupils  ─┬─< pickup_authorisations(pupil_id)        the people allowed to collect
         ├─< daily_pickup_codes   (pupil_id)
         └─< release_records      (pupil_id)
pickup_authorisations ─┬─< daily_pickup_codes (authorisation_id)
                       └─< release_records    (authorisation_id)
"A ─< B (col)" means: one A has many B, linked by B.col
```

IN PLAIN ENGLISH

Pickup people are not attached to a school directly. They belong to a pupil, and the pupil belongs to a school. That is why services join through the `pupils` table to check the school.

School (models/school.py → table `schools`)

Columns: `id`, `name` (150), `address` (255), `created_at`, `updated_at`. Created once, at registration.

User and UserRole (models/user.py → `users`)

`class UserRole(str, Enum)`: `PARENT="parent"`, `SCHOOL_ADMIN="school_admin"`, `GATE_STAFF="gate_staff"`. Stored with SQLAlchemy's `Enum` type, which creates a PostgreSQL enum type `userrole` holding the member *names* (PARENT, ...); the API always speaks the lower-case *values*.

Column	Notes
`id`, `school_id`	Every user belongs to exactly one school.
`name` (150), `phone` (30)	Contact details.
`email` (255, **unique**)	The sign-in identity; unique across all schools.
`password_hash` (255)	Argon2 hash; never the password.
`role`	A `UserRole`.
`password_changed_at`	Set on reset/change; tokens issued before it are rejected.
`deactivated_at`	Set when an admin deactivates; NULL means active.
`created_at`, `updated_at`	Automatic.

The `is_active` **property** (computed, not stored) returns `deactivated_at is None`.

Pupil (models/pupil.py → `pupils`)

Columns: `id`, `school_id`, `parent_id` (→ users), `first_name` / `last_name` (100 each), `photo_url` (500, optional), `date_of_birth` (Date), timestamps.

`parent: Mapped["User"] = relationship(lazy="raise")` lets code read `pupil.parent.name`, but only when the query loaded it explicitly (`selectinload`). `lazy="raise"` turns an accidental hidden query into an error, because async sessions cannot run hidden queries. `TYPE_CHECKING` imports `User` only for type checkers, avoiding a circular import.

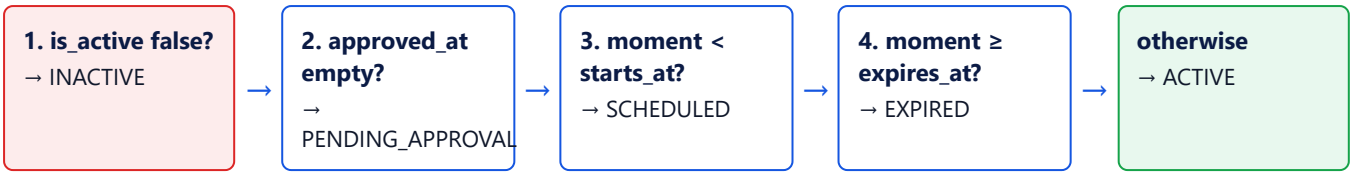
PickupAuthorisation and AuthorisationStatus (models/pickup_authorisation.py)

A "pickup person": someone a parent allows to collect a pupil, for a period.

Column	Notes
`pupil_id`	Which pupil they may collect.
`picker_name` (150), `picker_phone` (30), `relationship` (50), `photo_url` (500, optional)	Who they are. The photo helps gate staff.
`type` (50)	A free label such as "Regular" or "One-off".
`starts_at`, `expires_at`	The validity period.
`is_active`	Switched off by deactivation.

`approved_at`	When a school admin approved; NULL means waiting (SRS 9.2).
`created_at`, `updated_at`	`updated_at` doubles as a version stamp for safe approval (see the approval workflow).

Status is never stored; it is worked out. `status_at(moment)` checks, in this exact order:



The `status` property is `status_at(now)`. Because it is computed, a person automatically becomes expired at the right moment with no background job.

DailyPickupCode and PickupCodeCheckResult (models/pickup_code.py)

`PickupCodeCheckResult` lists every possible outcome of checking a code at the gate: `valid`, `invalid_code`, `wrong_pupil`, `wrong_authorisation`, `used_code`, `expired_code`, `authorisation_inactive`, `authorisation_pending`, `authorisation_scheduled`, `authorisation_expired`.

Table `daily_pickup_codes`: `pupil_id`, `authorisation_id`, `code_hash` (the HMAC, **unique index** for fast gate lookup), `code_date`, `expires_at`, `used_at` (NULL until released), `created_at`. A second index on (`authorisation_id`, `code_date`) finds today's codes when a new one is issued.

ReleaseRecord (models/release.py → `release_records`)

The permanent proof that a pupil left with someone (SRS 12): `pupil_id`, `authorisation_id`, `verified_by` (the gate staff user), `verification_method` (`qr` or `manual`), `result` (`ReleaseResult.RELEASED` = "released"), `released_at`, `created_at`. Index (`pupil_id`, `released_at`) for history pages.

SecurityAlert (models/security_alert.py → `security_alerts`)

`school_id`, optional `pupil_id`, `authorisation_id`, `attempted_by` (staff user), `alert_type` (a check result name, or `repeated_failed_attempts` / `pickup_refused`), `message` (never the code), `created_at`, `resolved_at` (NULL = open). Indexes: (`attempted_by`, `created_at`) to count a staff member's recent failures, and (`school_id`, `created_at`) for the alerts page.

AuditLog (models/audit_log.py → `audit_logs`)

`school_id`, `user_id` (who acted; optional), `action` (such as `pupil.created`), `entity_type` / `entity_id` (what was affected), `metadata_` (JSON; the Python name has a trailing underscore because `metadata` is reserved by SQLAlchemy, while the database column is still called `metadata`), `created_at`.

PasswordResetToken (models/password_reset_token.py)

`user_id`, `token_hash` (SHA-256, 64 characters, unique index), `expires_at`, `used_at`, `created_at`. Used both for "forgot password" and for the first "set your password" link of admin-created accounts.

16. app/schemas: what goes in and out

Schemas are **Pydantic models**: classes that describe the exact shape of request bodies and responses. FastAPI uses them to validate input (wrong type, missing field, too long → automatic 422), to convert output to JSON, and to draw the Swagger docs. They are separate from the database models on purpose: the API shape and the table shape can differ, and response schemas list only safe fields.

Common Pydantic tools you will see:

Tool	Meaning
`BaseModel`	Base class for a schema.
`Field(examples=[...])`	Extra info; here only examples for Swagger.
`ConfigDict(from_attributes=True)`	Allows building the schema straight from a database object (`Schema.model_validate(pupil)`).
`ConfigDict(extra="forbid")`	Unknown fields are an error (422) instead of being ignored silently.
`@model_validator(mode="after")`	A custom check that runs after the individual fields are validated.
`@computed_field` + `@property`	A value calculated from other fields and included in the JSON output.
`EmailStr`	Text that must be a valid email.
`AwareDatetime`	A timestamp that must include a time zone (naive times are rejected).
`model_dump(exclude_unset=True)`	Only the fields the client actually sent; used by PATCH updates.
`model_fields_set`	The names of the fields the client sent.

schemas/fields.py: shared field types

`_text(max_length)` returns a type "string, trimmed of spaces, at least 1 character, at most max_length". The limits match the database columns, so over-long input is a clear 422, never a database crash.

Type	Rule	Used for
`Name`	trimmed, 1–150	user names, school names, picker names
`PupilName`	1–100	pupil first/last names
`Phone`	1–30	phones
`Label`	1–50	relationship, type
`Address`	1–255	school address
`PhotoUrl`	1–500	photo links
`Password`	1–128, not trimmed, must contain a non-space	passwords (spaces inside are allowed)
`ResetToken`	1–128 of `A-Z a-z 0-9 _ -`	reset tokens

schemas/auth.py

Schema	Shape and rules
`UserRegister` (in)	name, email, phone, password, school_name, school_address: creates a school and its first admin.
`UserResponse` (out)	id, school_id, name, email, phone, role. **No password hash.**
`SchoolUserResponse` (out)	UserResponse + `is_active`, for admins.
`CurrentUserResponse` (out)	UserResponse + `school_name`, for "who am I".
`UserLogin` (in)	email, password.
`TokenResponse` (out)	access_token, token_type ("bearer").
`SchoolUserCreate` (in, extra forbidden)	name, email, phone, role. No password: the new user sets one via email.
`SchoolUserUpdate` (in, extra forbidden)	Optional name, email, phone; an explicit `null` is rejected by `reject_nulls` (leave a field out to keep it).
`OwnDetailsUpdate` (in, extra forbidden)	Optional name, phone only. Email and role cannot be self-changed.
`PasswordChange` (in)	current_password, new_password; validator rejects a new password equal to the current one.
`PasswordResetEmail`, `PasswordResetConfirm` (in)	email; token + new_password.
`MessageResponse` (out)	detail text.

schemas/pupil.py

`PupilCreate`: first_name, last_name, optional photo_url, date_of_birth, and `parent_id` (required for admins, forbidden for parents; enforced in the service). `PupilParent`: id, name, email, phone (so staff can contact the family). `PupilResponse`: the pupil plus its nested `parent`. `PupilUpdate`: all optional; nulls rejected except `photo_url` (sending `null` removes the photo).

schemas/pickup_authorisation.py

`normalise_to_utc` and `UtcDatetime`: a timestamp type that must carry a time zone and is converted to UTC. This is the project's rule for every timestamp input (a time without a zone is ambiguous, so it is a 422).

`PickupAuthorisationCreate` (pupil_id, picker details, type, starts_at, expires_at); `PickupAuthorisationUpdate` (all optional, nulls rejected except photo_url); `PickupAuthorisationApprove` (`updated_at`: the version the admin saw); `PickupAuthorisationResponse` (all columns plus the computed `status`).

schemas/pickup.py

Schema	Shape and rules
`PickupCodeRequest`	`authorisation_id`.

`PickupCodeResponse`	authorisation_id, pupil_id, **code** (plaintext, returned this one time), code_date, expires_at, and computed `qr_data`: compact JSON of code, code_date and expires_at for the client to draw as a QR code.
`VerificationMethod`	`qr` or `manual`.
`PickupVerifyRequest`	method, code (≤64), qr_data (≤512), optional pupil_id. Validator: QR needs only qr_data; manual needs only code.
`GatePupil`, `GatePicker`	The minimum shown at the gate: names, relationship, photos. No phone numbers or dates of birth.
`PickupVerifyResponse`	result, method, pupil/authorisation/code_expires_at **only when valid**, and computed `verified`. A failed check reveals nothing about whose code it was.
`PickupRefuseResponse`	result, method, computed `refused`.

schemas/release.py, alert.py, audit_log.py, school.py

- `PickupReleaseResponse`: result, method, and when released: release_id, released_at, pupil, authorisation; computed `released`.
- `ReleaseHistoryItem`: one history row with nested pupil, pickup person, `verified_by` (staff id and name, or `None` for parents, who are not told which staff member released their child, SRS 18.9), method, result, time.
- `AlertStatus` (open/resolved) and `SecurityAlertResponse` with computed `status`.
- `AuditActor` (id, name) and `AuditLogResponse`.
- `SchoolResponse`: id, name, address, timestamps.

17. app/routes: the HTTP endpoints

Each file creates an `APIRouter` with a `prefix` (e.g. `/pupils`) and a `tags` label (the heading in Swagger). `routes/__init__.py` imports the nine routers and includes them all into one `router` with prefix `/api/v1`, which `main.py` attaches to the app.

Anatomy of an endpoint

```
@router.post(                                  # method + path (after prefixes: POST /api/v1/pupils)
    "",
    response_model=PupilResponse,              # output is filtered/validated to this shape
    status_code=status.HTTP_201_CREATED,       # success code
    dependencies=[Depends(require_role(*MANAGER_ROLES))],   # role gate (403 otherwise)
)
async def create_pupil_endpoint(
    data: PupilCreate,                         # JSON body, validated (422 if bad)
    current_user: User = Depends(get_current_user),         # signed-in user (401 if not)
    session: AsyncSession = Depends(get_session),           # database session
):
    try:
        return await create_pupil(session=session, school_id=current_user.school_id, ...)
    except ValueError as exc:                  # business rule broken → chosen HTTP error
        raise HTTPException(status_code=422, detail=str(exc)) from exc
```

Several routers define `_parent_scope(current_user)`: it returns the user's own id when the caller is a parent, and `None` for staff. Services treat `None` as "whole school" and an id as "only this parent's pupils". This one line is how parents and admins share endpoints safely.

Many routers also define `_found(x)` / `_managed(x)`: return the object or raise 404. A record in another school is reported as "not found", never "forbidden", so its existence is not revealed.

Complete endpoint catalogue

Endpoint	Who	What it does
health.py (no prefix)		
`GET` `/health`	anyone	Process is running: `{"status":"ok"}`.
`GET` `/health/db`	anyone	Runs `SELECT 1`.
`GET` `/health/redis`	anyone	PINGs Redis; 503 if down.
`GET` `/health/auth`	signed in	Proves JWT auth works.
`GET` `/health/admin`	admin	Proves role checks work.
auth.py (prefix `/auth`)		
`POST` `/auth/register`	anyone (5/IP/h)	Create a school and its first admin. 409 if email taken.
`POST` `/auth/login`	anyone (limited)	Returns a JWT. 401 on bad credentials or deactivated account.

POST `/auth/password-reset/request`	anyone (limited)	Always 202 with the same message; emails a link if the account exists.
POST `/auth/password-reset/confirm`	anyone (limited)	Token + new password → 204, or 400 if invalid/used/expired.
POST `/auth/change-password`	signed in	Needs current password; returns a fresh token; signs out other sessions; emails a notice.
GET `/auth/me`	signed in	The user + school name (the web app uses this for role routing).
PATCH `/auth/me`	signed in	Change own name/phone.
school.py (prefix `/schools`), all admin only		
GET `/schools/me`	admin	The admin's school profile.
POST `/schools/users`	admin	Create a parent or gate staff; emails a set-password link. 409 on duplicate email.
GET `/schools/users?role=&limit=&offset=`	admin	List school users (limit 1–500, default 200), ordered by name.
PATCH `/schools/users/{id}`	admin	Change a parent's or gate staff's name/email/phone. Admins cannot be edited here.
POST `/schools/users/{id}/deactivate` · `/activate`	admin	Switch an account off/on.
pupils.py (prefix `/pupils`)		
POST `/pupils`	parent, admin	Create (parents for themselves; admins must give `parent_id`).
GET `/pupils` · `/pupils/{id}`	parent, admin	List / read (parents see only their own).
PATCH `/pupils/{id}`	parent, admin	Edit details.
pickers.py (prefix `/pickers`): pickup authorisations		
POST `/pickers`	**parent only**	Add a pickup person (starts unapproved). Emails admins in the background.
GET `/pickers?pupil_id=` · `/pickers/{id}`	parent, admin	List / read.
PATCH `/pickers/{id}`	parent, admin	Edit. A parent's edit clears the approval.
POST `/pickers/{id}/activate` · `/deactivate`	parent, admin	Switch on/off. A parent re-activating needs approval again.
POST `/pickers/{id}/approve`	admin	Approve, sending the `updated_at` seen. 409 if already approved or changed since.
pickup.py (prefix `/pickup`): codes and the gate		

`POST` `/pickup/codes`	parent (10/h)	Issue today's code for an active authorisation. 201 with plaintext code + qr_data.
`POST` `/pickup/verify`	gate staff (30/min)	Check a code; always 200 with `verified` true/false. Failures raise alerts.
`POST` `/pickup/release`	gate staff (30/min)	Re-check and release: 201 + release record, or 200 with `released: false`.
`POST` `/pickup/refuse`	gate staff (30/min)	Person does not match: cancel the code, raise a `pickup_refused` alert.
releases.py, **alerts.py**, **audit_logs.py**		
`GET` `/releases?pupil_id=&since=&until=&limit=&offset=`	parent, admin	Release history, newest first; `since` inclusive, `until` exclusive; 422 if until ≤ since.
`GET` `/alerts?status=open|resolved` · `/alerts/{id}`	admin	Security alerts.
`POST` `/alerts/{id}/resolve`	admin	Resolve; 409 if already resolved.
`GET` `/audit-logs?action=&entity_type=&entity_id=`	admin	Read-only audit trail, newest first.

Paths above omit the `/api/v1` prefix. Lists take `limit` (max 200 unless noted) and `offset` for paging.

Route-specific details worth knowing

- **auth.py** `_client_ip` reads `request.client.host`. Behind a proxy, Uvicorn fills this from `X-Forwarded-For` only for trusted proxies (`FORWARDED_ALLOW_IPS`), which is why the deployment checklist matters for per-IP limits.
- **login**: `hit` the IP counter, `check` the per-email lock, try to log in; on failure log the IP only, `record` a failure against the email, answer 401.
- **request_reset**: answers 202 with `PASSWORD_RESET_SENT` whether or not the account exists, and sends the email via `background_tasks` after the response, so timing does not reveal anything either.
- **change_own_password**: wrong current passwords are counted per user (`pikva:rl:change-password:user:<id>`), so a stolen session cannot be used to guess the password.
- **pickers.py** `_tell_admins_if_pending`: after a parent creates, edits or activates a person who is now pending, schedules `notify_approval_requested(school_id)` as a background task.
- **pickup.py** `limit_gate_staff` / `limit_parent_codes`: rate limits written as dependencies, listed in `dependencies=[...]` next to the role check.
- **pickup.py** `_email_admins_later`: builds a small `notify(school_id, alert_ids)` function that queues `notify_security_alerts` as a background task. The service calls it only after its commit, so the email job reads alerts that really exist.
- **release endpoint** changes `response.status_code` to 201 only when `result.released` is true.
- **alerts.py** and **audit_logs.py** put the admin role check on the whole router (`APIRouter(dependencies=[...])`), so no endpoint in them can forget it.

18. app/services: the business rules

Services receive plain values (a session, the school id, the parent scope, validated data, the acting user's id), do the work, write audit records, and commit. They never see HTTP. Errors are signalled by returning `None` (not found / not accessible) or raising `ValueError` (rule broken).

> **WHY IT IS BUILT THIS WAY**
>
> **The school-scoping pattern.** Look for helpers named `_scoped_pupils` and `_scoped_authorisations`. Every read starts from one of them, so it is impossible to write a pupil query in these services that forgets the school filter. When you add a feature, start from these helpers too.

audit_service.py

Defines every audit action name as a constant (`USER_CREATED = "user.created"`, `PUPIL_RELEASED = "pupil.released"`, ... 19 in total) and `FORBIDDEN_METADATA_KEYS` (access_token, code, password, password_hash, qr_data, secret_key, token).

- `_find_forbidden_key(metadata)`: searches nested dictionaries and lists for a forbidden key name.
- `async record_audit(session, school_id, user_id, action, entity_type, entity_id=None, metadata=None)`: refuses (ValueError) metadata containing a forbidden key, adds an `AuditLog` row and flushes. **Does not commit**: the caller commits it together with the change it describes, so an action and its audit record always succeed or fail together.
- `async list_audit_logs(...)`: school's records joined (outer join) to the acting user's name, optional filters, newest first.

auth_service.py

- `register_school_admin(session, data)`: 409-style ValueError if the email exists; creates the `School`, flushes (to get its id), creates the admin `User` with a hashed password, audits `user.created` (via registration), commits.
- `login_user(session, data)`: unknown email → dummy verify then "Invalid email or password"; wrong password → same message; **only after a correct password** is a deactivated account reported, so a guesser learns nothing. Returns `{"access_token": ..., "token_type": "bearer"}`.
- `update_own_details(session, user, data)`: applies only fields that actually changed; audits the field *names* (never values); commits.

school_service.py

- `CreatedSchoolUser` dataclass: the new user and the set-up email to send after commit.
- `get_school_by_id`.
- `create_school_user(session, school_id, data, actor_id)`: only parent/gate staff; duplicate email check (and again via `IntegrityError` if two requests race); unusable password; issues a password-reset token; audits; builds the set-up email *before* committing (while objects are loaded); commits.
- `list_school_users`: school filter, optional role, ordered by name.
- `get_managed_user`: a parent or gate staff in this school, or None. Admins are never "managed".

- `update_school_user`: changed fields only; email uniqueness check; audits field names.
- `set_school_user_active`: sets or clears `deactivated_at`; asking for the current state changes nothing; audits.

pupil_service.py

- `create_pupil`: if the caller is a parent, the pupil is theirs and they may not send `parent_id`; if admin, `parent_id` is required and must be a parent in the same school. Audits, commits, then re-reads so the response includes the parent summary.
- `_scoped_pupils(school_id, parent_id)`: `select(Pupil)` + `selectinload(Pupil.parent)` + school filter (+ parent filter).
- `list_pupils` (ordered by last name, first name), `get_pupil`, `get_parent_pupil_by_id`.
- `update_pupil`: sets only sent fields, audits field names, commits, re-reads.

pickup_authorisation_service.py

- `create_pickup_authorisation`: pupil must belong to this parent; expiry must be after start; created with `approved_at = None` (pending); audits.
- `_scoped_authorisations`: joins `pupils` to filter by school (and parent).
- `list_pickup_authorisations`, `get_pickup_authorisation`.
- `update_pickup_authorisation`: validates the resulting start/expiry pair; applies changes; `_reset_approval_if_parent` clears `approved_at` if a **parent** changed an approved person (so an approved grandmother cannot be quietly swapped for a stranger); audits with `approval_reset`.
- `set_pickup_authorisation_active`: a parent switching a person back *on* also needs approval again, so a parent cannot undo an admin's deactivation.
- `approve_pickup_authorisation(..., seen_updated_at, ...)`: one atomic `UPDATE ... WHERE id=? AND approved_at IS NULL AND updated_at = seen RETURNING id`. If nothing was updated, it re-reads to explain: "already approved" or "has changed; review it again". This is **optimistic concurrency**: the admin can only approve exactly the version they looked at.

pickup_service.py: codes and gate checks

Constants: `MAX_CODE_ATTEMPTS = 5`; `AlertNotifier` type; `_AUTHORISATION_FAILURES` maps each non-active status to its check result (a test makes sure every status is listed, otherwise an unlisted status would count as valid). `PickupCodeCheck` dataclass holds the result and the matched code and authorisation.

Function	What it does
`issue_pickup_code`	Finds the parent's authorisation; **locks its row** (`refresh(..., with_for_update=True)`) so two simultaneous requests cannot both end with a valid code; requires status ACTIVE; expiry = the earlier of next UTC midnight and the authorisation's end; expires any unused code issued today for it; generates a code and inserts its hash inside a **savepoint** (`begin_nested`), retrying on the astronomically rare hash collision; audits (without the code); commits; returns the plaintext once.

`code_from_qr_data`	Parses the scanned JSON and returns only `code`; the date and expiry inside the QR are ignored because the database decides validity.
`check_pickup_code`	Looks up the code by hash, joined to its authorisation and pupil, **within the staff member's school** (other schools' codes look invalid). Then in order: wrong pupil → wrong authorisation → used → expired → authorisation status → valid.
`mark_code_used`	Atomic `UPDATE ... SET used_at=now WHERE id=? AND used_at IS NULL AND expires_at > now AND pupil in school RETURNING id`. Only one gate can win. Does not commit.
`check_presented_code`	Chooses QR or manual input, then calls `check_pickup_code`.
`record_failed_check`	Records alerts (linked to the code's real pupil/authorisation, never to client-sent ids), logs ids and result names only, audits `pickup.verification_failed`. Returns alert ids. Does not commit.
`expire_code_now`	Atomically sets an unused, unexpired code's expiry to now (used by refusal).
`refuse_pickup`	If the code is valid and can be expired: add a `pickup_refused` alert, audit, commit, notify admins. If it became used/expired in the meantime, record it as a normal failed check instead.
`verify_pickup`	Failure: record, commit, notify, return the result only. Success: return the minimal pupil and picker details. **Changes nothing** on success; the release is a separate step.

release_service.py

- `release_pupil`: re-checks the code (it may have been used at another gate since verification), then `mark_code_used`. If that loses the race, the outcome becomes `used_code` or `expired_code` and is recorded as a failure. On success: creates the `ReleaseRecord`, audits `pupil.released`, and commits **all three together**: code used + release record + audit. A pupil is never released without a record, and a code can release only once.
- `list_release_history`: joins release → pupil → authorisation → staff user; school filter (+ parent filter); optional pupil and half-open time range; newest first; hides `verified_by` from parents.

alert_service.py

Constants: `REPEATED_FAILURE_LIMIT = 5` within `REPEATED_FAILURE_WINDOW` = 10 minutes; types `repeated_failed_attempts` and `pickup_refused` are "urgent"; `ALERT_EMAIL_COOLDOWN_SECONDS` = 10 minutes; `_FAILURE_MESSAGES` gives human text per result (never containing the code).

- `record_failed_verification`: adds one alert typed by the failure; then counts this staff member's alerts in the last 10 minutes by type (excluding refusals); if there are 5 or more and no repeated-attempts alert yet in the window, adds a `repeated_failed_attempts` alert. Does not commit.
- `refused_pickup_alert`: builds (does not add) the refusal alert.
- `list_alerts`, `get_alert`; `resolve_alert` uses a conditional atomic update so only one of two simultaneous resolutions wins (the other gets 409).
- `security_alert_email`: subject and plain text listing alerts by **id only** (no names, no codes).

- `_claim_email_slot` / `_release_email_slot`: Redis `SET key 1 NX EX 600` marks "an email was sent to this school recently". Urgent alerts are sent anyway. If Redis is down, send (a missed alert is worse than an extra email). If sending fails, the slot is released so the next alert retries.
- `notify_security_alerts(school_id, alert_ids)`: runs as a background task **after the response**, so it opens its own session (`async_session()`), loads this school's alerts and admin emails, applies the cooldown, sends, and never raises.

approval_notification_service.py

The same pattern for "a pickup person awaits approval": `approval_request_email` (names nobody; links to `WEB_APP_URL/admin/approvals`), a 15-minute per-school cooldown in Redis, `notify_approval_requested` run in the background.

notification_service.py: sending email

- `EmailMessage` (frozen dataclass): `to` (tuple), `subject`, `text`.
- `EmailSender` (a `Protocol`): "anything with `async send(message)`". This lets tests pass a fake sender.
- `LogEmailSender`: logs the subject and recipient count only. Used when Resend is not configured.
- `ResendEmailSender`: POSTs JSON to `https://api.resend.com/emails` with the API key as Bearer token, 10-second timeout; `raise_for_status()` turns non-2xx into an error.
- `get_email_sender()`: Resend if both key and from-address are set, otherwise the log sender.
- `send_email_safely(message, sender=None) -> bool`: never raises; logs failures by error type or HTTP status only.

password_reset_service.py

- `request_password_reset(session, email)`: unknown or deactivated → `None` (the route answers identically); otherwise `issue_password_reset_token`, audit, commit, return the token for the email.
- `issue_password_reset_token(session, user)`: cancels the user's open tokens, stores only the new token's hash with a 30-minute expiry, flushes.
- `_reset_link(token)` → `WEB_APP_URL/reset-password?token=...`.
- `password_reset_email`, `account_setup_email`: the only two emails that carry a secret (the single-use, expiring link).
- `reset_password(session, token, new_password)`: atomic `UPDATE ... SET used_at WHERE hash matches AND unused AND unexpired RETURNING user_id` (a link works once even if clicked twice at the same moment); sets the new hash and `password_changed_at` (signing out all sessions); cancels other tokens; audits; commits.
- `change_password(session, user, current, new)`: verifies the current password, sets the new one and `password_changed_at`, cancels reset links, audits, builds the notice email, commits, and returns a **new** token created after the change so this device stays signed in.

19. Alembic: migrations

`alembic.ini · alembic/env.py · alembic/versions/`

`alembic.ini` holds Alembic's own settings (script location, logging). `env.py` is run by every Alembic command. It:

1. imports `Base` and **every model module** (so all tables are registered in `Base.metadata`);
2. sets the database URL from `settings.database_url` (so Alembic uses the same `.env` as the app);
3. sets `target_metadata = Base.metadata`, which lets `alembic revision --autogenerate` compare the models with the real database and draft a migration;
4. runs migrations either "offline" (prints SQL) or "online" through an async engine (`run_async_migrations` → `connection.run_sync(do_run_migrations)`).

Each file in `versions/` has a `revision` id, a `down_revision` (the one before it, forming a chain), an `upgrade()` and a `downgrade()`. The chain, oldest first:

#	Revision	Change
1	`0457ff04e08e`	Initial schema: schools, users, audit_logs, pupils, pickup_authorisations, daily_pickup_codes, release_records, security_alerts.
2	`9bd124be07d5`	Pickup authorisation timestamps become time-zone aware.
3	`0d4845d275c5`	All remaining timestamps become time-zone aware.
4	`1f8129deaebb`	Daily pickup code indexes (unique hash; authorisation + date).
5	`45e9c6f2653f`	Security alert index on (attempted_by, created_at).
6	`e5014ab3c0a5`	Release history index.
7	`615f0b7ec554`	Alert and audit list indexes.
8	`3c8d1f7a92b4`	`password_reset_tokens` table and `users.password_changed_at`.
9	`7b2e4c9d1a60`	`pickup_authorisations.approved_at` (school approval).
10	`4f1c8e2b7d93` (head)	`users.deactivated_at`.

WATCH OUT

Never edit a migration that has already been applied to a real database. Add a new one instead: change the model, run `uv run alembic revision --autogenerate -m "what changed"`, **read and correct** the generated file, then `uv run alembic upgrade head`. Autogenerate misses some things (renamed columns, enum changes, data fixes).

20. The test suite

`BACKEND/tests`

Tests are small programs that call the code and assert the result. They run against a real PostgreSQL database named `pikva_test` and Redis database 15, never your development data.

tests/conftest.py, the shared set-up

- Before the app is imported, it rewrites `settings.database_url` to `pikva_test` and `settings.redis_url` to database 15, and blanks the Resend key so no real email is ever sent.
- It swaps the Argon2 hasher for the cheapest settings, so tests run fast (hashes still verify because Argon2 stores its settings in each hash).
- `test_database` (once per run): refuses to run unless connected to `pikva_test`, migrations have been applied, and test Redis answers.
- `clean_tables` (after every test): `TRUNCATE` all tables, so each test starts empty. `clean_redis`: `FLUSHDB` on database 15.
- `client`: an `httpx.AsyncClient` with `ASGITransport(app=app)`, which calls the app in-process with real HTTP semantics but no network.

What the test files cover

Files	Area
`test_security_api`, `test_session_revocation`, `test_password_hashing`	Auth, cross-school isolation, token rejection after password change/deactivation.
`test_login_rate_limit`, `test_register_rate_limit`, `test_gate_rate_limit`, `test_code_rate_limit`, `test_rate_limit`	Every rate limit, including fail-open behaviour.
`test_pickup_code_service`, `test_pickup_code_api`, `test_pickup_code_security`, `test_pickup_verify_input`, `test_pickup_refuse_api`, `test_pickup_schemas`	Code issue, every check result, concurrency, release, refusal.
`test_pickup_approval_service`, `test_pickup_approval_api`, `test_approval_notification`	School approval and its email.
`test_alert_service`, `test_notification_service`, `test_audit_service`	Alerts, repeated attempts, cooldowns, email sending, forbidden audit keys.

`test_password_reset_service`, `test_password_reset_api`, `test_change_password_api`, `test_own_details_api`	Account security flows.
`test_school_users_api`, `test_school_user_management_api`, `test_pupils_api`	User and pupil management.
`test_release_schemas`, `test_release_date_filter`, `test_request_schemas`, `test_input_limits`	Validation and history filters.
`test_health`, `test_openapi_docs`, `test_production_config`, `test_env_example`, `test_logging_config`, `test_operational_logging`	Operations: health checks, docs hidden in production, key checks, logs free of secrets.

scripts/e2e_school.py

Used by the web app's browser tests. `create <state.json>` writes a school named with a random `e2e-` tag, an admin, a parent, a gate staff member and a pupil straight into the configured database and saves their sign-in details to the JSON file. `cleanup <state.json>` deletes only that school's rows, children first, in one transaction, after checking the tag. Both refuse to run when `ENVIRONMENT=production`.

Part C · End-to-end workflows

Real journeys traced through every layer

Legend: User person, Browser , Next.js web server (Volume 2), API FastAPI, DB PostgreSQL, Redis , Email Resend.

The browser never calls FastAPI directly. Next.js keeps the JWT in a cookie the browser's scripts cannot read and forwards each call with `Authorization: Bearer`. The backend steps are the same whoever the client is (web, future mobile app, tests).

21. A school signs up

1. API `POST /auth/register` with name, email, phone, password, school name and address. Pydantic validates every field.
2. Redis `hit("pikva:rl:register:ip:<ip>", 5, 3600)`: the 6th attempt in an hour gets 429.
3. DB `register_school_admin`: email unused? Insert `schools` row, flush, insert `users` row (role SCHOOL_ADMIN, Argon2 hash), insert audit `user.created`, commit.
4. API 201 with the safe `UserResponse`. The admin can now sign in.

22. Signing in and using the API

1. User types email and password on `/login`; Next.js posts them to `POST /api/v1/auth/login` with the user's IP in `X-Forwarded-For`.
2. Redis IP counter (30 per 15 min) and per-email failure lock (5) are checked.
3. DB user loaded by email; Argon2 verify in a worker thread; deactivated accounts refused only after a correct password.
4. API returns `{"access_token": "eyJ...", "token_type": "bearer"}`, valid 30 minutes. On failure: 401, failure recorded in Redis.
5. Next.js stores the token in an httpOnly cookie and calls `GET /auth/me` to learn the role, then sends the user to `/admin`, `/gate` or `/parent`.
6. Every later call carries the token; `get_current_user` re-checks signature, expiry, user existence, password-change time and active status **on every request**.

23. An admin adds a parent (or gate staff)

1. API `POST /schools/users` {name, email, phone, role} (admin only; unknown fields such as "password" → 422).
2. DB `create_school_user`: role must be parent/gate staff; email unused; user created with an **unusable** password; a reset token issued (hash stored, 30 min); audit; commit.
3. Email after the 201 response, a background task sends "Set up your Pikva account" with `WEB_APP_URL/reset-password?token=...`.
4. User opens the link and chooses a password → `POST /auth/password-reset/confirm` → token atomically marked used, password hashed, `password_changed_at` set, audit, 204. They can now sign in.

24. Adding a pupil

An admin sends `POST /pupils` with `parent_id` (must be a parent in the same school); a parent sends it without `parent_id` (the pupil becomes theirs). The service audits `pupil.created` with the parent id and returns the pupil with a nested parent summary.

25. A parent adds a pickup person; the school approves

1. API `POST /pickers` (parent only) {pupil_id, picker_name, picker_phone, relationship, type, photo_url?, starts_at, expires_at}. Times must include a time zone; they are stored in UTC.
2. DB pupil must be this parent's; expiry after start; row inserted with `approved_at = NULL` → status **pending_approval**; audit.
3. Email background task: if no approval email went to this school in the last 15 minutes (Redis `SET NX EX`), all the school's admins get "a pickup person is waiting for approval" with a link to the Approvals page. No names in the email.
4. Admin reviews on `/admin/approvals`; the page holds the person's `updated_at`.
5. API `POST /pickers/{id}/approve` {updated_at}. DB one conditional UPDATE: approve only if still unapproved **and** unchanged since the admin looked. Otherwise 409 with the reason.
6. Status becomes **active** (or scheduled/expired depending on the dates). Declining is done by `/deactivate`.
7. If the parent later edits the person (or re-activates them), `approved_at` is cleared and the cycle repeats.

26. A parent gets today's pickup code

1. API `POST /pickup/codes` {authorisation_id}. Role parent; Redis limit 10 per hour per parent.
2. DB authorisation loaded through the parent scope, then **locked** (`SELECT ... FOR UPDATE`) and its status re-read; must be ACTIVE, else 422 "Pickup authorisation is pending_approval" (etc.).
3. Expiry = min(next midnight UTC, authorisation end). Any unused code from today for this person is expired now: **only the newest code works**.
4. 8 random characters generated; HMAC-SHA256 hash inserted in a savepoint (retry up to 5 times on collision); audit `pickup_code.generated` with ids and dates (never the code); commit.
5. API 201 {code: "7F3B9K2D", code_date, expires_at, qr_data: "{\"code\":...}"}. This is the only time the plaintext exists anywhere; the parent's screen draws the QR from `qr_data`.

27. At the gate: verify, then release or refuse

Present
Adult shows code or QR.

→

Verify
`POST /pickup/verify`. Nothing changes on success.

→

Look
Staff compare the person with the photos/names shown.

→

Release
`POST /pickup/release`: code used + record + audit, one commit.

Or refuse
`POST /pickup/refuse`: code expired at once + `pickup_refused` alert + audit; admins emailed.

•

Any failed check
Alert typed by reason + audit; 5 failures by one staff member in 10 minutes adds a `repeated_failed_attempts` alert; admins emailed (10-min cooldown, urgent types always).

1. API role gate_staff; Redis budget 30 calls/minute per staff member (verify, refuse and release share it).
2. Body: `{"method":"manual","code":"7F3B 9K2D"}` or `{"method":"qr","qr_data":"{...}"}`. QR data is parsed and only its `code` used.
3. DB `check_pickup_code`: hash the code, find it **in this school only**, then the ordered checks (pupil, authorisation, used, expired, authorisation status).
4. Verify success returns pupil and picker names/photos and the code's expiry. Verify failure returns only the result name (e.g. `used_code`) and records the alert.
5. Release re-runs the check and then the atomic `mark_code_used`. If two gates press Release on the same code at once, exactly one UPDATE matches; the other becomes `used_code`.
6. Email background task after the response: load the committed alerts, check the Redis cooldown, email the school's admins (ids only).

28. After the pickup: history, alerts and audit

- **History**: `GET /releases` joins release, pupil, pickup person and staff. Parents see only their pupils and never the staff member; admins see everything in the school. The web app passes `since` / `until` computed in the viewer's time zone for "this day".
- **Alerts**: admins list open alerts and resolve them (`POST /alerts/{id}/resolve`, audited, 409 if someone else already did).
- **Audit log**: `GET /audit-logs` shows who did what and when, with field names but no personal values or secrets.

29. Password reset, password change and deactivation

- **Forgot password**: `/password-reset/request` always answers 202 the same way; if the account exists and is active a link is emailed; older links are cancelled. Confirm marks the token used atomically and sets `password_changed_at`, which makes every older JWT fail in `get_current_user`.
- **Change password**: needs the current password (failures counted per user); returns a new token for this device; others are signed out; a notice is emailed.
- **Deactivation**: `deactivated_at` set; login refused; existing tokens rejected on the very next request; no reset links issued.

30. Security measures at a glance

Threat	Defence (where)
Stolen database reveals passwords/codes	Argon2 passwords, HMAC codes, SHA-256 reset tokens (`security.py`).
Password or code guessing	Redis rate limits, per-email lock, gate budget, repeated-attempt alerts.
Discovering which emails exist	Same login message, dummy hash timing, identical reset answer.
Seeing another school's data	School filter in every service query; other schools' records are 404 / invalid_code.
Wrong role doing an action	`require_role` on every endpoint or router.
Code used twice / race at two gates	Atomic conditional UPDATEs; row lock when issuing.
Approving a changed person unseen	`updated_at` version check; parent edits clear approval.
Old sessions after reset/deactivation	`iat` vs `password_changed_at` ; `is_active` check per request.
Secrets in logs, audit or email	Forbidden audit keys, ids-only logging and emails.
Weak production configuration	Start-up key check; docs hidden in production.

Part D · Owning the code

Run it, test it, change it, ship it

31. Running the backend locally

```
# one-time: install uv and Docker Desktop, then from BACKEND/
docker compose up -d                      # PostgreSQL on 5432, Redis on 6379
cp .env.example .env                      # then edit DATABASE_URL and SECRET_KEY
uv sync                                   # install dependencies into .venv
uv run alembic upgrade head               # create/upgrade tables
uv run fastapi dev app/main.py            # http://127.0.0.1:8000, auto-reload
```

Open `http://127.0.0.1:8000/docs` for Swagger: every endpoint with a "Try it out" button. Use **Authorize** with a token from `/auth/login`. `BACKEND/MANUAL_TESTING.md` walks through every feature by hand.

> **TRY IT YOURSELF**
>
> Register a school in Swagger, log in, paste the token into Authorize, then call `GET /api/v1/auth/me`. Now call `GET /api/v1/alerts` with no token and see the 401/403. Then set `DATABASE_ECHO=true`, restart, and watch the SQL each call produces.

32. Running the tests

The tests need a separate, migrated database named `pikva_test` and Redis running. For example:

```
docker exec -it pikva-postgres createdb -U postgres pikva_test     # once
# migrate it: point DATABASE_URL at pikva_test for this one command (environment beats .env)
DATABASE_URL=postgresql+asyncpg://postgres:postgres@127.0.0.1:5432/pikva_test uv run alembic upgrade
head
uv run pytest                     # all tests
uv run pytest tests/test_pickup_code_api.py -k release -x   # one file, matching names, stop at first
failure
```

Re-run the migration command on `pikva_test` whenever you add a migration.

33. Recipe: adding a feature safely

Example: "admins can add a note to a pupil".

1. **Model**: add `note: Mapped[str | None] = mapped_column(String(500), nullable=True)` to `Pupil`.
2. **Migration**: `uv run alembic revision --autogenerate -m "add pupils.note"`; read it; `upgrade head` on both databases.
3. **Schema**: add a constrained type (e.g. `_text(500)` in `fields.py`) and the field to `PupilUpdate` and `PupilResponse`.
4. **Service**: if only admins may set it, check the role in the service or route; keep using `_scoped_pupils`; audit the field *name*.
5. **Route**: usually unchanged for an extra field. For a new endpoint, add `require_role`, a `response_model`, and map `ValueError` / `None` to status codes.
6. **Tests**: success, wrong role (403), other school (404), invalid input (422).
7. **Frontend**: update the TypeScript type and page (Volume 2), and if it is a new top-level path, check the proxy's allowed paths.

> **WATCH OUT**
> Checklist for every new query: is it filtered by `school_id`? Does a parent get only their own records? Does a failure path leak whether a record exists? Are secrets kept out of logs, audit and email?

34. Deploying

Follow `BACKEND/DEPLOYMENT.md` line by line. The essentials: a dedicated PostgreSQL user and backups; Redis reachable only from the API host; `ENVIRONMENT=production`; a generated 48-byte `SECRET_KEY`; Resend domain verified; `WEB_APP_URL` on https; `uv sync --frozen` and `alembic upgrade head`; run `uv run fastapi run app/main.py --workers N` behind an HTTPS reverse proxy with `FORWARDED_ALLOW_IPS` set correctly (including the web host); then check `/health`, `/health/db`, `/health/redis` and that `/docs` returns 404.

35. Troubleshooting

Symptom	Likely cause and fix
Start-up error: field required `database_url` / `secret_key`	No `.env` in the folder you ran from, or the variable missing.
`RuntimeError: SECRET_KEY ...`	Production with a weak key: generate a new one.
Connection refused on 5432 / 6379	`docker compose up -d`; check `docker ps`.
`relation "..." does not exist`	Migrations not applied: `uv run alembic upgrade head`.
Tests exit "Run Alembic migrations against pikva_test"	Create and migrate `pikva_test` (Part D, tests).
Every request 401 after a while	Token expired (30 min) or `SECRET_KEY` changed. Sign in again.
429 during manual testing	Rate limit; wait `Retry-After` seconds or `docker exec pikva-redis redis-cli FLUSHDB` (development only).

Emails never arrive	Resend not configured (they are only logged), or sender domain unverified.
Parent cannot get a code (422 "... pending_approval")	The pickup person needs admin approval, or is scheduled/expired/inactive.

36. Glossary

ASGI
The standard interface between async Python web servers (Uvicorn) and frameworks (FastAPI).

Atomic update
A single database statement that checks and changes a row in one indivisible step, so two simultaneous requests cannot both succeed.

Audit log
Permanent, read-only record of who did what and when.

Background task
Work FastAPI runs after sending the response (emails).

Bearer token
A token sent as `Authorization: Bearer <token>`; whoever holds it is treated as the user.

Commit / flush / rollback
Finalise a transaction / send pending changes without finalising / undo.

Dependency (FastAPI)
A function an endpoint declares with `Depends`; FastAPI runs it and passes its result.

Enum
A fixed set of named values.

Fail open
When a safety service is down, allow the action rather than block everyone.

Foreign key
A column that must match another table's primary key.

Hash / HMAC
One-way fingerprint / fingerprint mixed with a secret key.

Index
A database structure that makes lookups on a column fast.

JWT
Signed token carrying the user id and expiry.

Migration
A versioned script that changes the database structure.

Optimistic concurrency
Accept a change only if the record is still the version the user saw (`updated_at`).

ORM

Maps tables to classes and rows to objects (SQLAlchemy).

Pydantic model / schema
A class describing and validating a data shape.

Rate limit
A cap on attempts per time window.

Row lock (FOR UPDATE)
Makes other transactions wait before changing the same row.

Savepoint
A mini-transaction inside a transaction that can be rolled back alone.

Tenant isolation
Keeping each school's data invisible to other schools.

UTC
The single world reference time zone; all Pikva timestamps are stored in it.